

FALCUTÉ DES SCIENCES ET TECHNOLOGIES

(L3-FST)

Rapport du travail

Cours : Systèmes

Étudiant : MACILLON Josias Yvenson

Professeur : SAINT AMOUR Ismaël

Le 19/10/ 2026

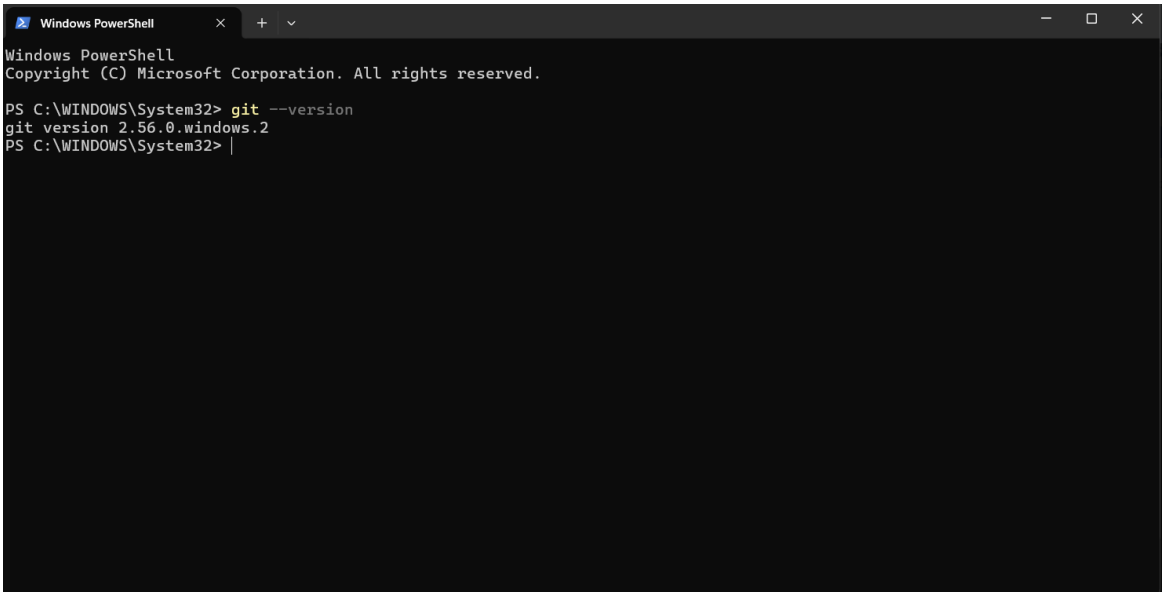
L’objectif de ce TD est de :

- Avoir une meilleure vue de Powershell et Git
- Maitriser les concepts de base de Git (système de gestion de versions).
- Apprendre et reproduire sur Github avec Powershell
- Maitriser l’hebergement sur Github

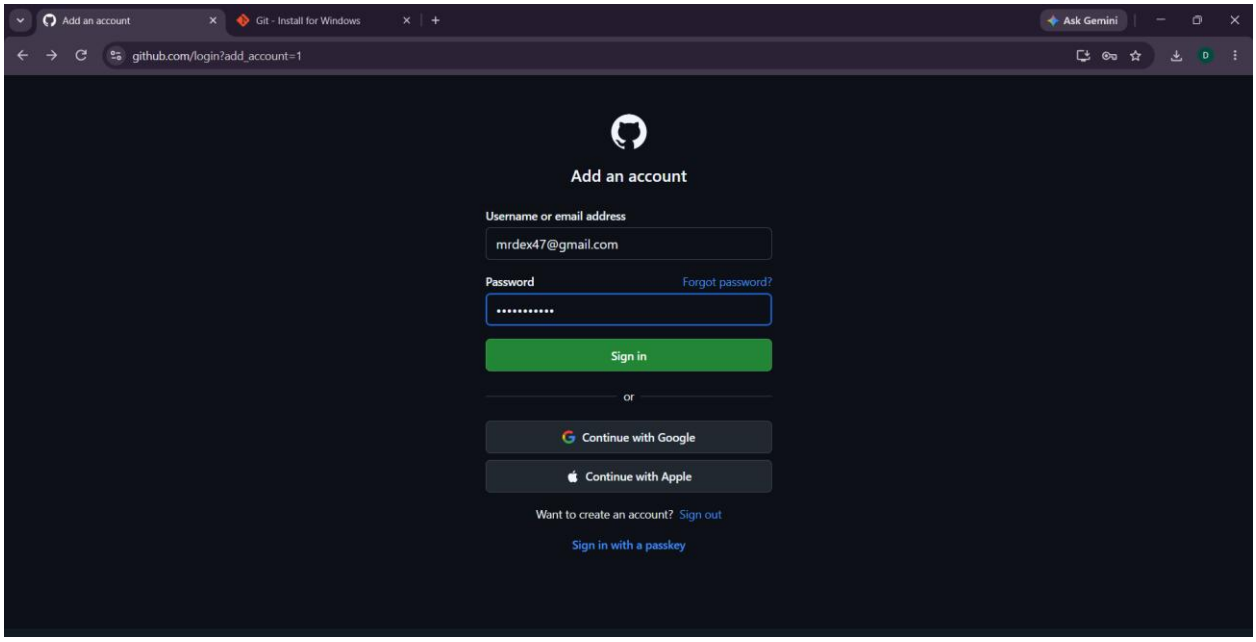
Travail Dirigé

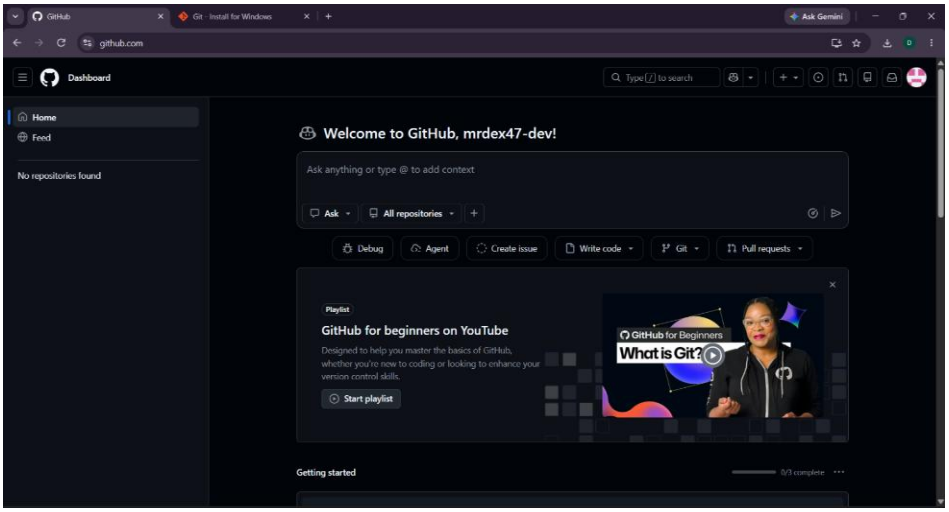
- Reproduire une série de tâches.

1) Vérification de la version Git installée sur le PC.

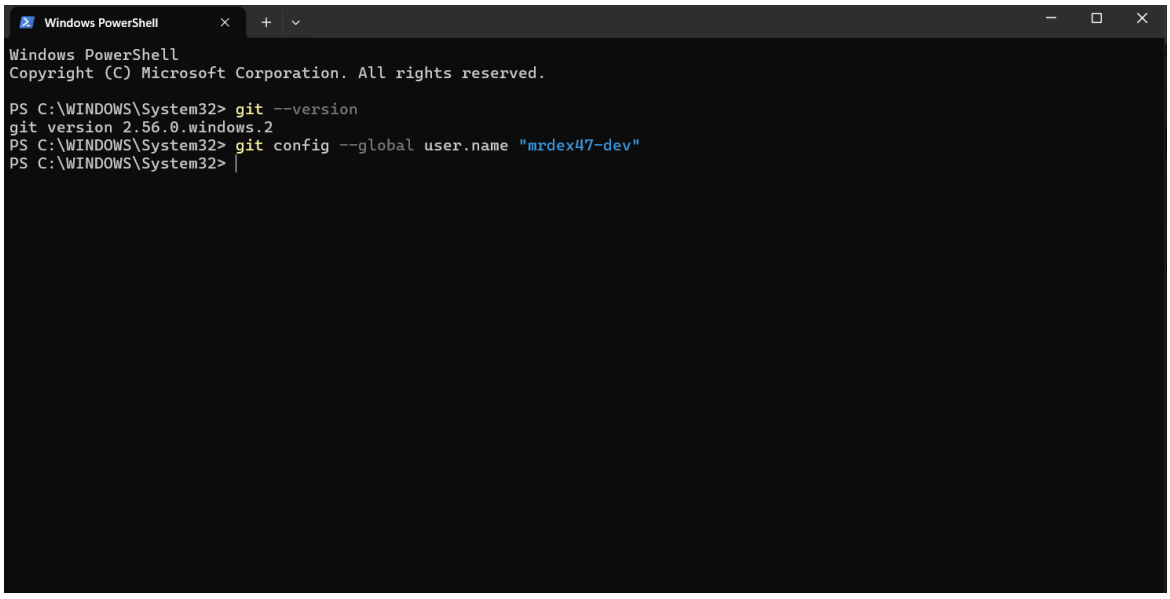


2) Deuxième étape : Connexion à mon compte GitHub

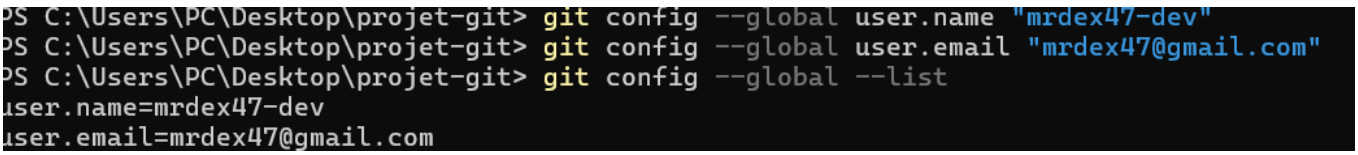




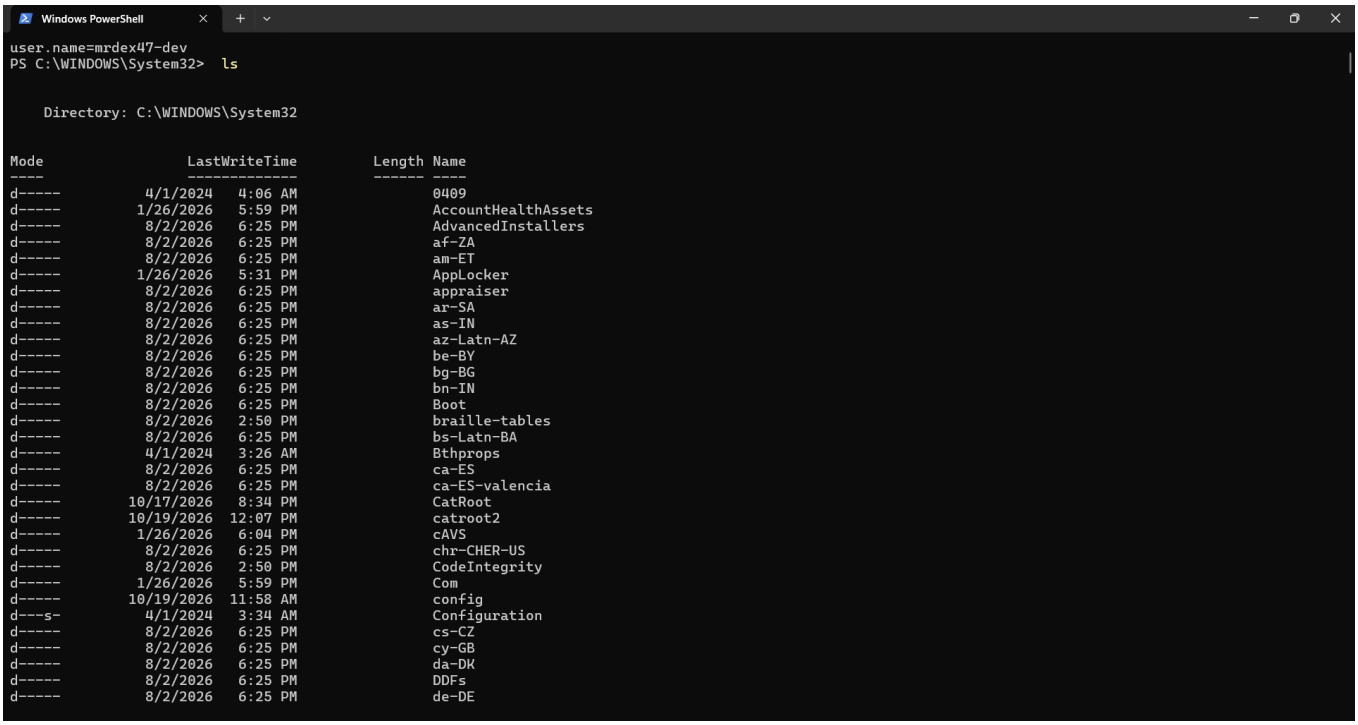
3) Troisième étape : Configuration de Git



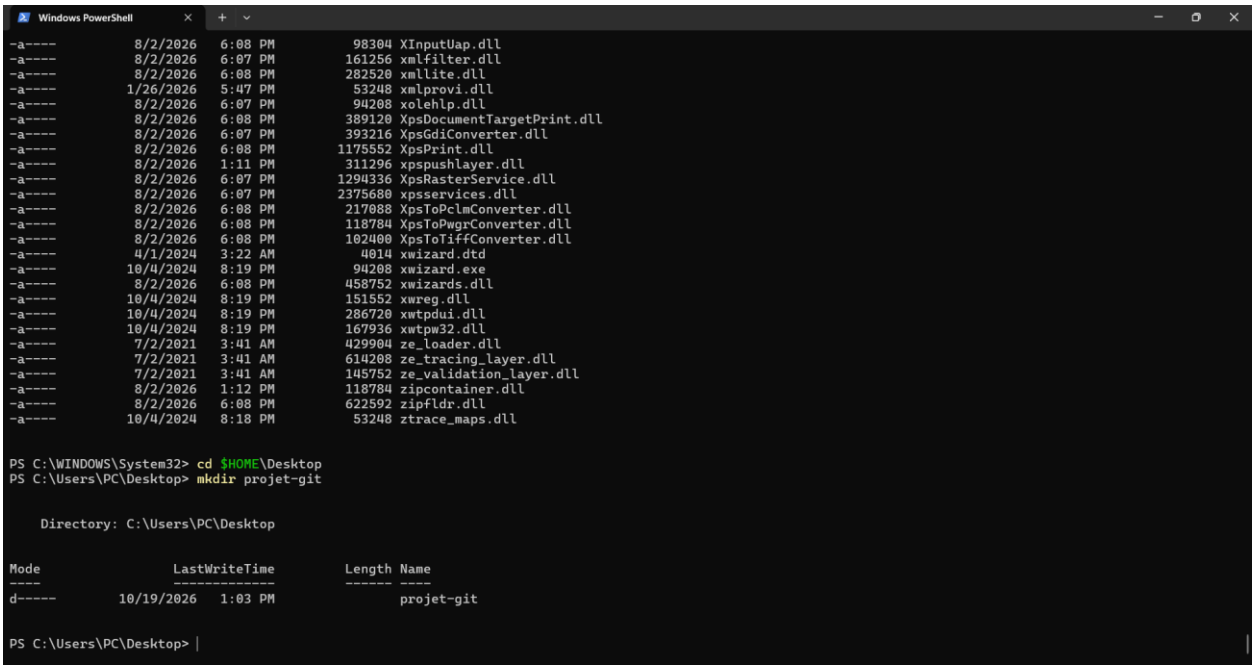
4)Quatrième étape : Vérification des données utilisateurs

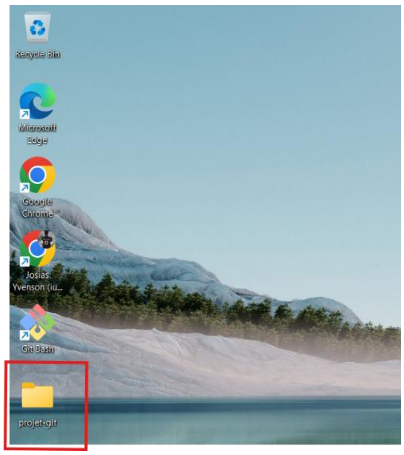


5) Cinquième étape : Affichage de la liste des fichiers et dossiers avec la commande ls



6) Sixième étape : Création d’un dossier dans Desktop





7) Septième étape : Entrée dans notre dossier « projet1-git » fraichement créé.

```
PS C:\WINDOWS\System32> cd $HOME\Desktop
PS C:\Users\PC\Desktop> mkdir projet-git


    Directory: C:\Users\PC\Desktop


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        10/19/2026   1:03 PM                projet-git


PS C:\Users\PC\Desktop> cd  projet-git
```

8) Huitième étape : Initialisation du dépôt git.

```
Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        10/19/2026   1:03 PM                projet-git

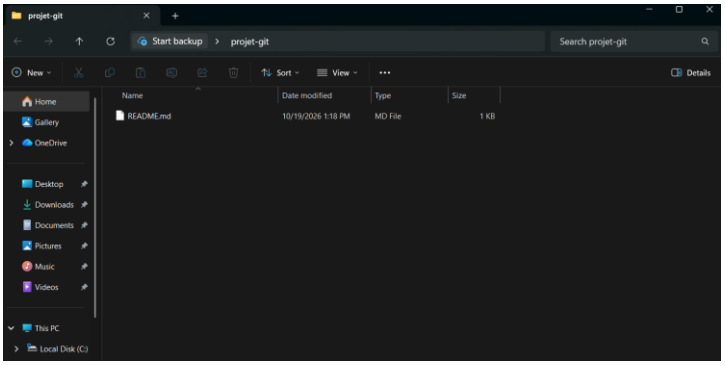

PS C:\Users\PC\Desktop> cd  projet-git
PS C:\Users\PC\Desktop\projet-git> git init
```

9) Neuvième étape : Ouverture et écriture de mon projet dans le fichier README.md

```
PS C:\Users\PC\Desktop\projet-git> git init
Reinitialized existing Git repository in C:/Users/PC/Desktop/projet-git/.git/
PS C:\Users\PC\Desktop\projet-git>  echo "Hello Git!" > README.md
PS C:\Users\PC\Desktop\projet-git>  git add README.md
PS C:\Users\PC\Desktop\projet-git> git commit -m "Premier commit avec Powershell"
[master (root-commit) 00e7c85] Premier commit avec Powershell
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 README.md
PS C:\Users\PC\Desktop\projet-git> git status
On branch master
nothing to commit, working tree clean
PS C:\Users\PC\Desktop\projet-git>
```

10)Dixième étape : Ajout du dossier README.md dans Git et premier commit effectué


```
PS C:\Users\PC\Desktop\projet-git> git init
Reinitialized existing Git repository in C:/Users/PC/Desktop/projet-git/.git/
PS C:\Users\PC\Desktop\projet-git>  echo "Hello Git!" > README.md
PS C:\Users\PC\Desktop\projet-git>  git add README.md
PS C:\Users\PC\Desktop\projet-git> git commit -m "Premier commit avec Powershell"
[master (root-commit) 00e7c85] Premier commit avec Powershell
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 README.md
PS C:\Users\PC\Desktop\projet-git> git status
On branch master
nothing to commit, working tree clean
PS C:\Users\PC\Desktop\projet-git>
```

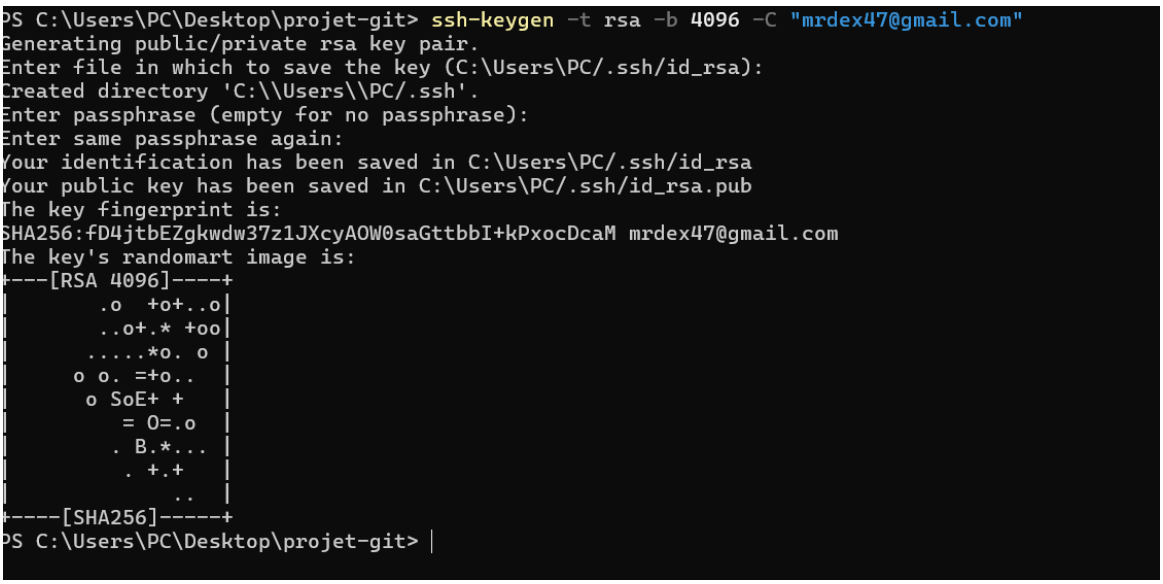



11) Onzième étape : Affichage de l'historique des commits dans un dépôt Git.

```
PS C:\Users\PC\Desktop\projet-git> git log
commit 00e7c85807f21e26f6d4748ae8678c68ecdcf125 (HEAD -> master)
Author: mrdex47-dev <mrdex47@gmail.com>
Date:   Mon Oct 19 13:18:21 2026 -0400

    Premier commit avec Powershell
PS C:\Users\PC\Desktop\projet-git>
```

12) Douzième étape : Génération d'une clé SSH



13) Treizième étape : Affichage de la clé privée avec la commande cat

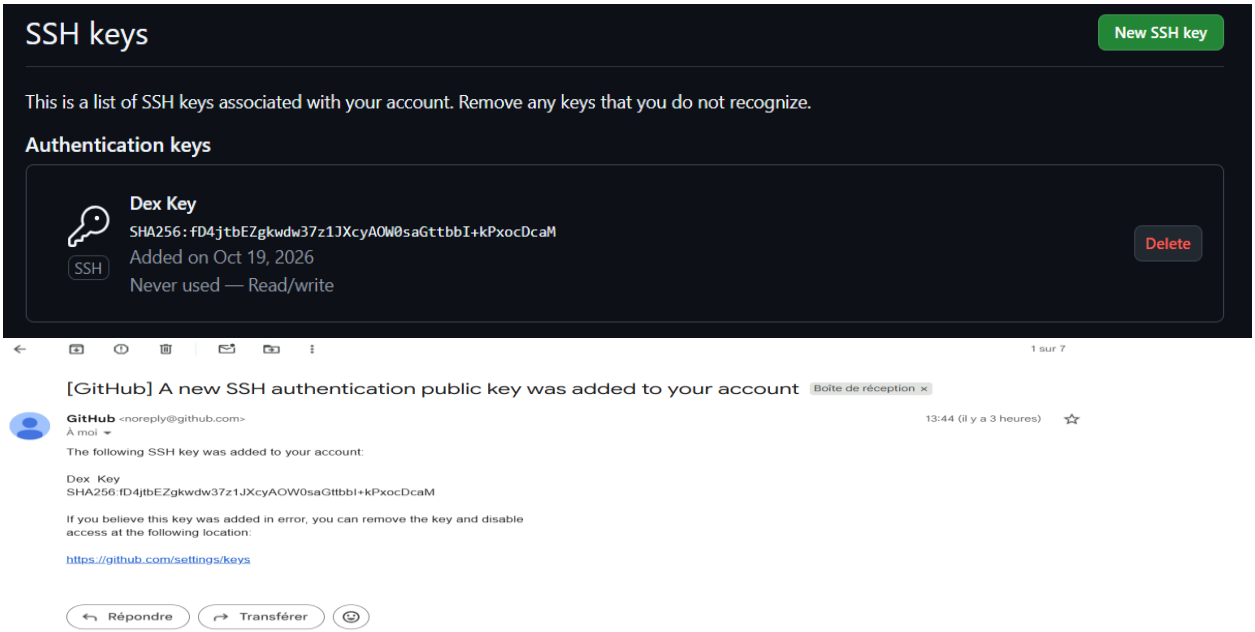




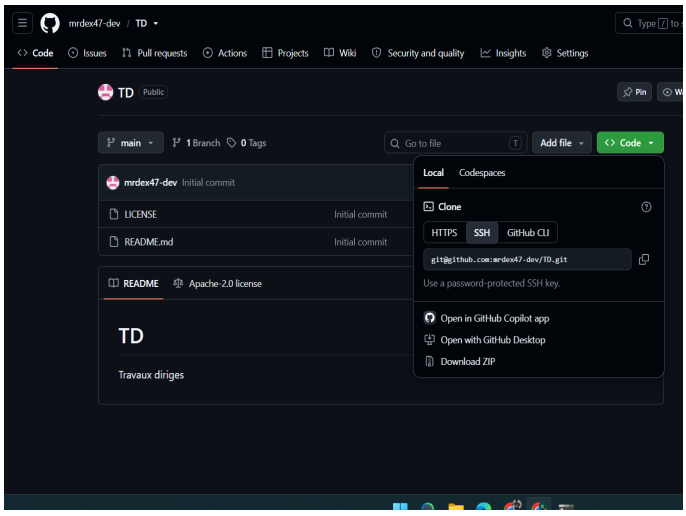
14) Quatorzième étape : Affiche de la clé publique

```
PS C:\Users\PC\Desktop\projet-git> cat ~/.ssh/id_rsa.pub
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAACAQCzL+Ch/hoosZUikMHXZv3EdqJhSC5YfM24QBPQqoiqNjGLbWXb7vgV0gudBsP64CCqe7wXnLZ7X/1dD9/2cAgIkMg4vqwK/32OjtpXUZhBosiiSqiC7fuj
Q+fxbLRVsCrilIzfpPBo++tmLeVZEuJjs6t86qYnDEjP8W6w/pq/sdm98f3cBWiMYXdUVake4ZVnBrPvdf+HeuQCctOTkOlO7xKoRw0U4QD3p/WQN3E9p72iQTc7bjFmpueH/QbQeqqMxq2FJZIUeelqJyTg
CF1UH0/oaK8xmjQ1zGBBVuKEvQNJmmOr3gdUnCZC4rVf/O5d7NKPein8q39StVUjie9rOqUW7tfkDLarDnuRYSWRXq5FOA7Kc/+4MLoTjMBUq3rt+Zx2gGWTImowyV6+HkHOnqBt3G+mwTrEFerJMSASMwws
EILQ5iywzQZ8IPFG/uyswXEesGgTeKfBTkMyxzc/ojkNNQ2iSM6XAy501geop7RyK+bYyzC37hGHGQ1EFt+CS4x3PzsAwoqITMVHZEydIWgwr1afWbsb6dRldccmXD2lxX9Y6qey526sHlu60urCx1jfWJoo
9LAihak+fk/dHqRKAxJAYN/h5Izpwvy2Y6zemuMZ5dq+8zWG6lFO0MCitdc3rDBgdYDdA3b71El0XB14D++QsQ1C7eXUJu7obow== mrdex47@gmail.com
```

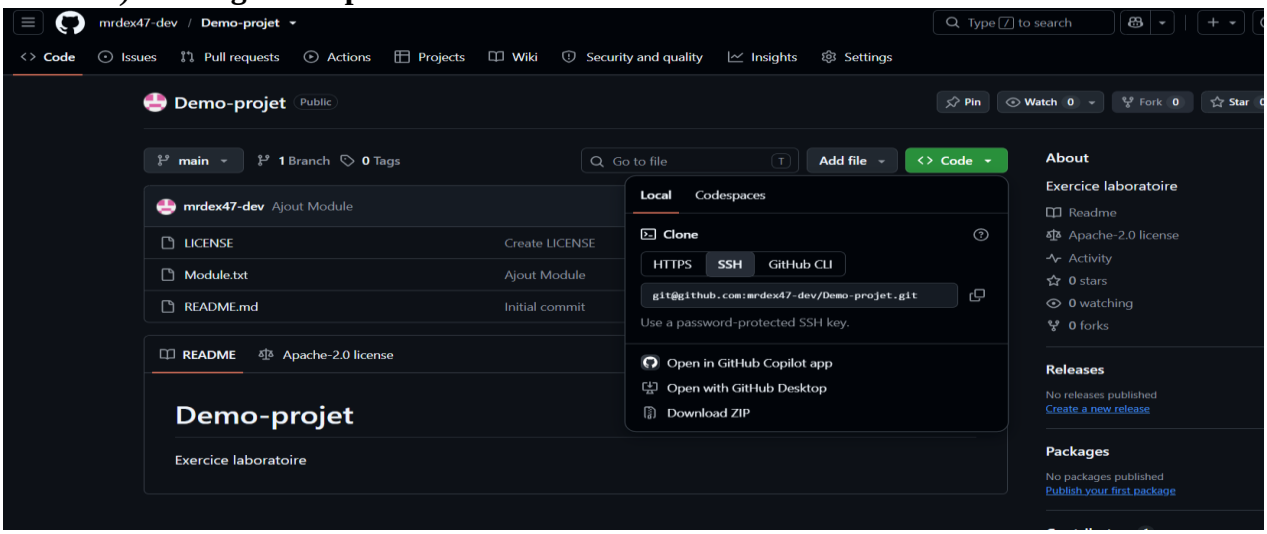
15) Quinzième étape : Ajout réussie de la clé sur mon compte GitHub



16) **Seizième étape : Création du dépôt GitHub**



17) **Clonage du dépôt GitHub via la clé SSH**




```
PS C:\Users\PC\Desktop> git clone git@github.com:mrdex47-dev/Demo-projet.git
Cloning into 'Demo-projet'...
remote: Enumerating objects: 6, done.
remote: Counting objects: 100% (6/6), done.
remote: Compressing objects: 100% (4/4), done.
remote: Total 6 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (6/6), done.
```

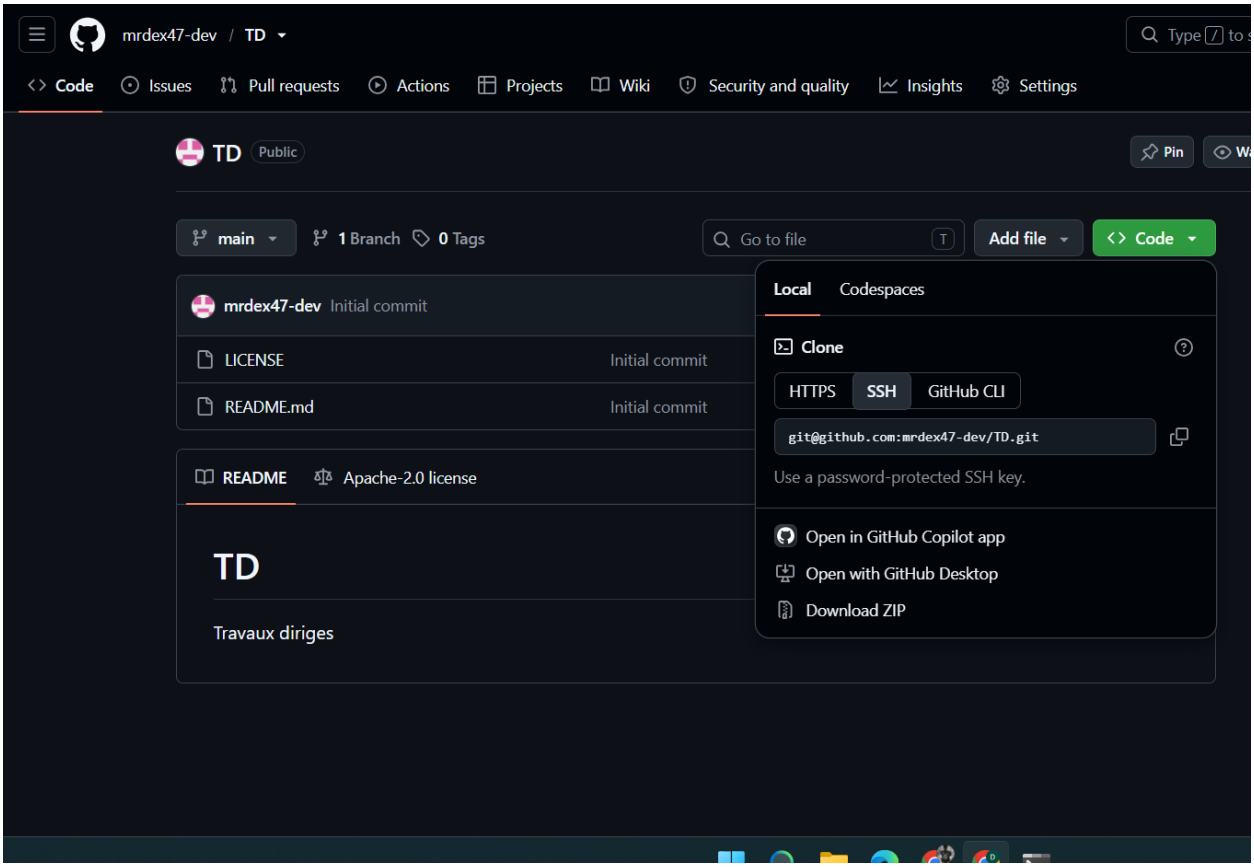
```
PS C:\Users\PC\Desktop> cd projet-git
PS C:\Users\PC\Desktop\projet-git> cd ..
PS C:\Users\PC\Desktop> cd demo-projet
PS C:\Users\PC\Desktop\demo-projet> echo "Systeme d'exploitation!" > Module.txt
PS C:\Users\PC\Desktop\demo-projet>  git add .
PS C:\Users\PC\Desktop\demo-projet>  git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   Module.txt

PS C:\Users\PC\Desktop\demo-projet> git commit -m "Ajout Module"
[main a38fcac] Ajout Module
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 Module.txt
PS C:\Users\PC\Desktop\demo-projet> git branch -M main
PS C:\Users\PC\Desktop\demo-projet> git push -u origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 4 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 353 bytes | 176.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:mrdex47-dev/Demo-projet.git
   f0164b8..a38fcac  main -> main
branch 'main' set up to track 'origin/main'.
PS C:\Users\PC\Desktop\demo-projet> git push -u origin main
branch 'main' set up to track 'origin/main'.
Everything up-to-date
```

Deuxieme partie

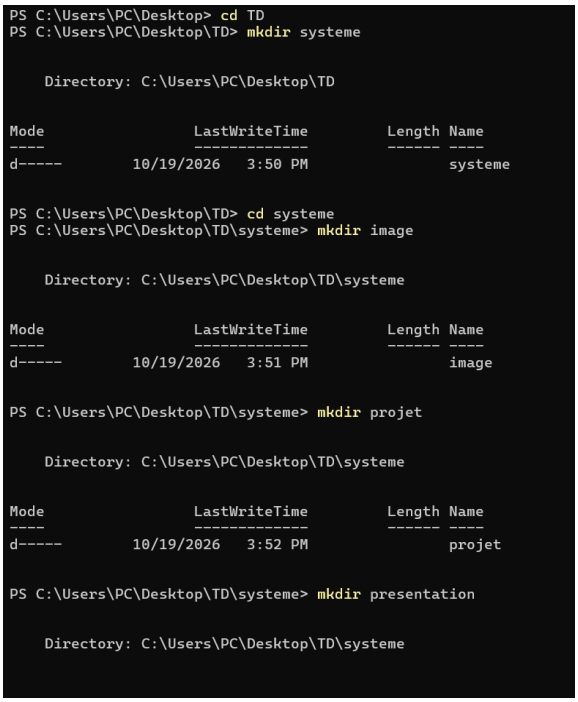
✓ Creer un dépôt GitHub nommé TD



✓ Clonage du dossier TD sur mon bureau

```
PS C:\Users\PC\Desktop> git clone git@github.com:mrdex47-dev/TD.git
Cloning into 'TD'...
remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (4/4), done.
PS C:\Users\PC\Desktop>
```

☐ Création des dossiers systeme, image, projet, presentation dans le dossier principal TD





Conclusion

4) Ce que j'ai retenu:

En réalisant ce TD, j'ai appris à comprendre quelques notions de base de GitHub, de Git et de PowerShell. J'ai également compris comment cloner un projet sur mon ordinateur, puis comment héberger un projet sur GitHub à l'aide de PowerShell.
J'ai aussi appris à utiliser certaines commandes Git et PowerShell, ainsi que leur utilité. Voici les principales commandes que j'ai retenues :

- **git --version** : permet de vérifier la version de Git installée sur l'ordinateur.
- **git config --global user.name et git config --global user.email** : permettent de configurer le nom d'utilisateur et l'adresse e-mail utilisés par Git pour identifier l'auteur des commits.
- **git config --list** : permet d'afficher la liste des configurations de Git.
- **ls** : permet d'afficher la liste des fichiers et des dossiers présents dans le répertoire actuel.
- **cd** : permet de naviguer entre les dossiers.
- **mkdir** : permet de créer un nouveau dossier.
- **git init** : permet d'initialiser un dépôt Git dans un dossier.
- **README.md** : est un fichier texte rédigé en Markdown qui sert généralement à présenter un projet et à expliquer son fonctionnement.
- **git log** : permet d'afficher l'historique des commits dans un dépôt Git.
- **git status** : permet d'afficher l'état actuel du dépôt Git, notamment les fichiers modifiés ou ajoutés.
- **git add .** : permet d'ajouter tous les fichiers modifiés et non suivis du répertoire courant à la zone de préparation des prochains commits.
- **git commit -m "message"** : permet d'enregistrer les modifications préparées dans un commit, accompagné d'un message descriptif.

- **git branch -M main** : permet de renommer la branche actuelle en main, en remplaçant une branche portant déjà ce nom si nécessaire.

En résumé, ce TD m’a permis de mieux comprendre le fonctionnement de Git et de GitHub, ainsi que l’utilisation de PowerShell pour gérer les fichiers, suivre les modifications d’un projet et faciliter son hébergement sur GitHub.